WEEK 2 INTERNSHIP REPORT

Project Title

Student Grade Calculator – Debugging and Troubleshooting Python Applications

1. Introduction

As part of my Week 2 internship task, I developed and debugged a Python-based Student Grade Calculator. The primary objective was to identify programming errors, understand their causes, implement appropriate solutions, and improve application reliability.

The application accepts a student's name and marks, calculates the average, and assigns a grade based on the calculated result.

2. Objectives

- Identify common Python programming errors.
- Understand runtime errors and their causes.
- Implement debugging techniques.
- Improve input validation.
- Handle exceptions using try-except.
- Test the application with different inputs.

3. Technologies Used

- Programming Language: Python
- Code Editor: Visual Studio Code
- Operating System: Windows
- Version Control: Git and GitHub

4. Project Implementation

The application consists of three major functions.

A. Average Calculation

The calculate_average() function calculates the average of the student's marks.

Code:

```
def calculate_average(marks):
```

```
    total = 0

    for mark in marks:
        total += mark

    return total / len(marks)
```

Design Decision:

A separate function was created for calculating averages to improve code readability and reusability.

B. Grade Calculation

The calculate_grade() function assigns grades based on the average.

Code:

```
def calculate_grade(average):
    if average >= 90:
        return "A"
    elif average >= 75:
        return "B"
    elif average >= 50:
        return "C"
    else:
        return "F"
```

Design Decision:

Conditional statements were selected because they provide a simple and understandable way to assign grades according to predefined ranges.

C. Input Validation and Exception Handling

Code:

```
try:
```

```
    marks_input = input("Enter marks: ")
    marks = [float(mark.strip())
            for mark in marks_input.split(",")]

except ValueError:
    print("Error: Please enter numeric marks only.")
    return

if not all(0 <= mark <= 100 for mark in marks):
    print("Error: Marks must be between 0 and 100.")
    return
```

Design Decision:

The try-except block prevents the application from terminating unexpectedly when users enter alphabetic characters.

The all() function ensures that every mark is within the permitted range.

5. Bugs Identified and Solutions

Bug 1: TypeError

Problem:
The program attempted to add a string to an integer.

Error:
TypeError: unsupported operand type(s) for +=: 'int' and 'str'

Solution:
Converted the input marks into numerical values using float().

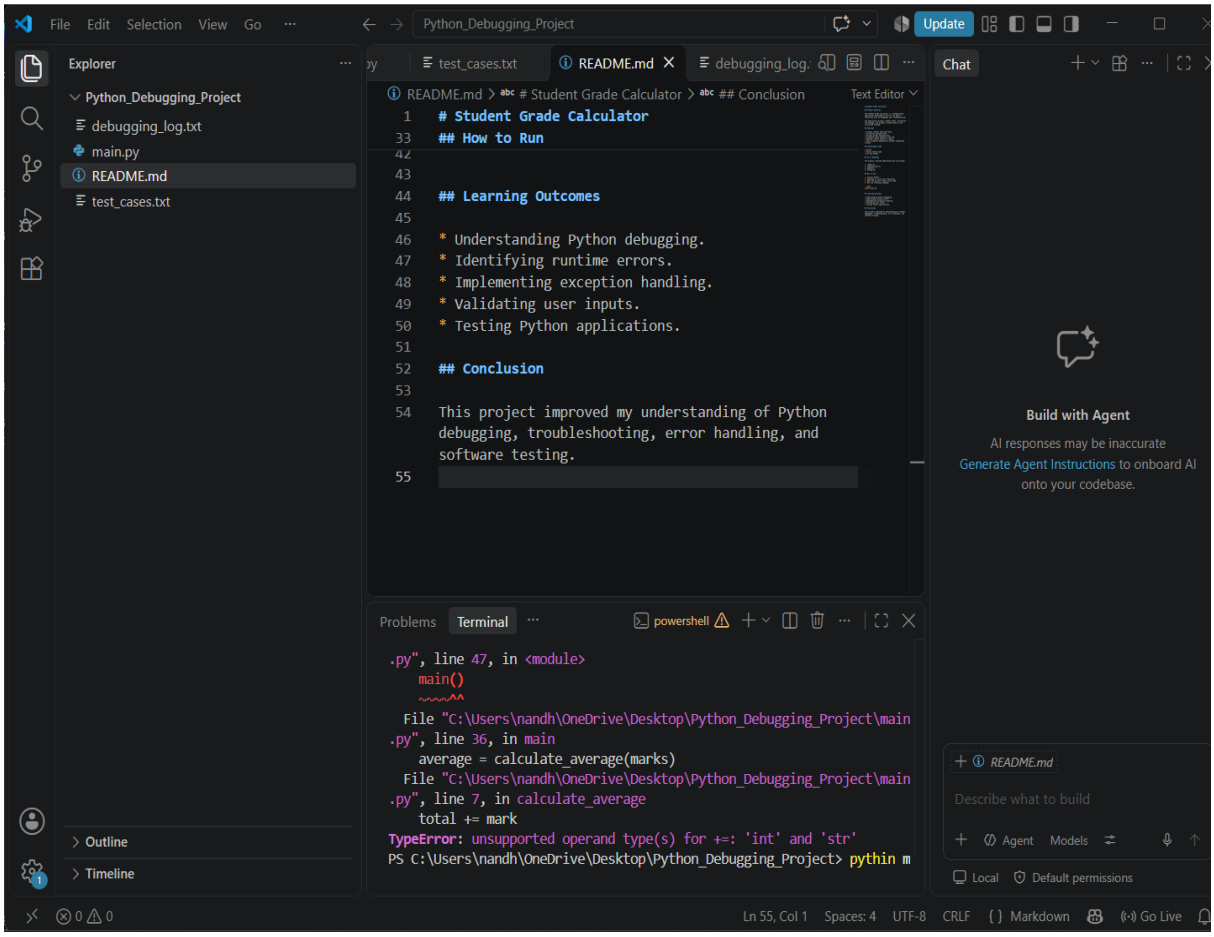


Figure 1:

TypeError encountered during initial execution

Bug 2: IndentationError

Problem:
The program contained inconsistent indentation.

Solution:
Corrected the indentation and maintained four spaces for nested code blocks.

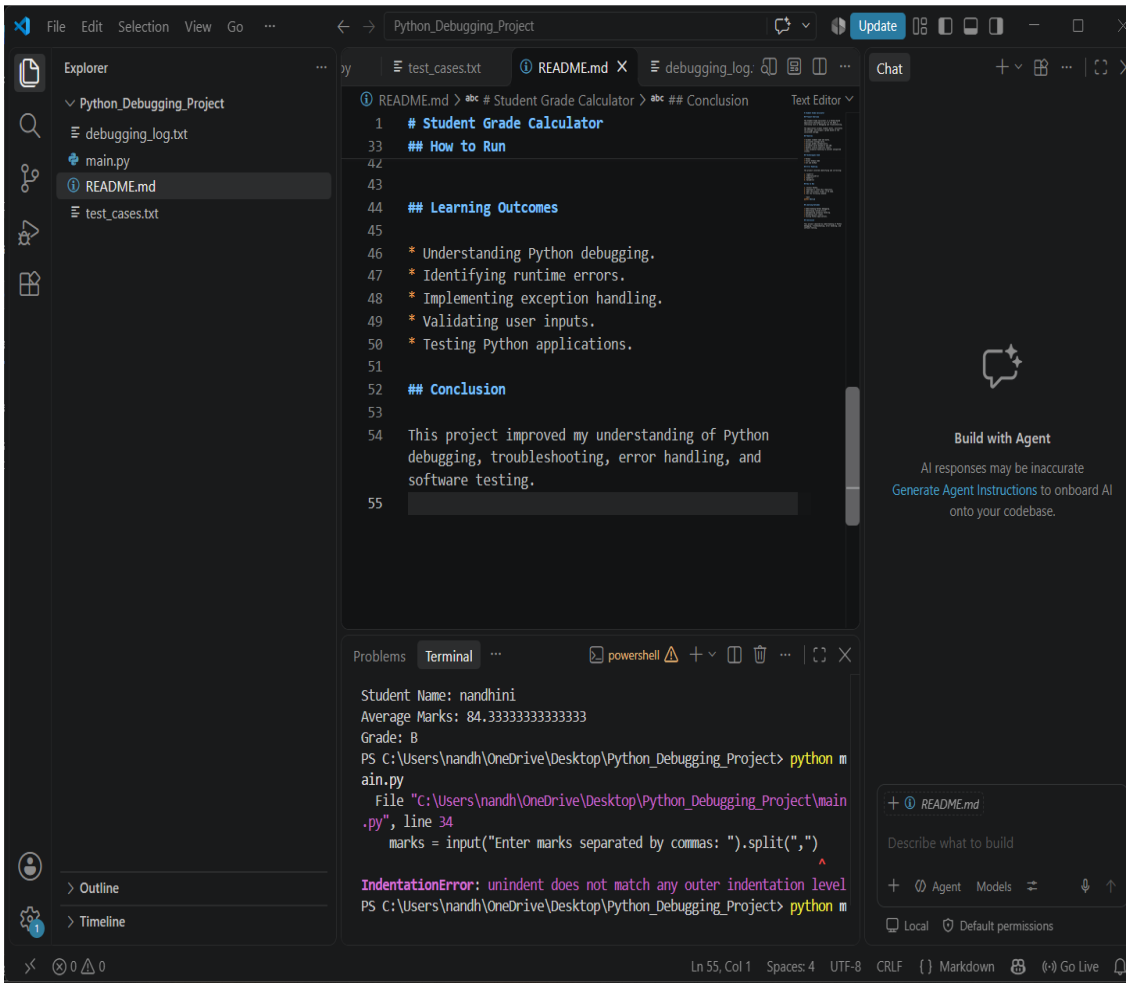


Figure 2:

IndentationError – Identification of inconsistent code indentation.

Bug 3: NameError

Problem:
The calculate_average() function was missing when it was called.

Solution:
Restored the function definition before its invocation.

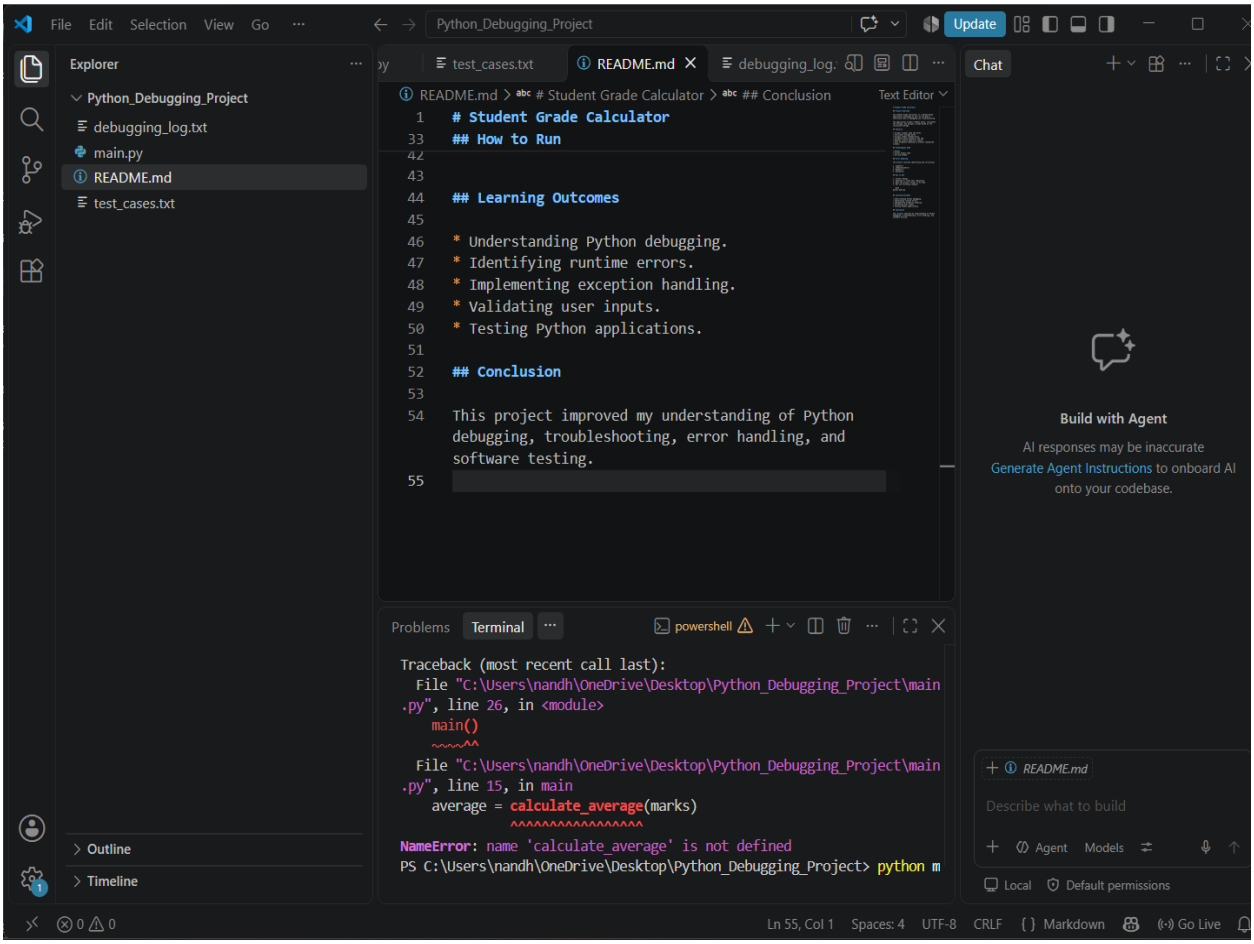


Figure 3:

NameError – Identification of an undefined function.

Bug 4: ValueError

Problem:
The program could crash when alphabetic characters were entered instead of numerical marks.

Solution:
Implemented exception handling using try-except.

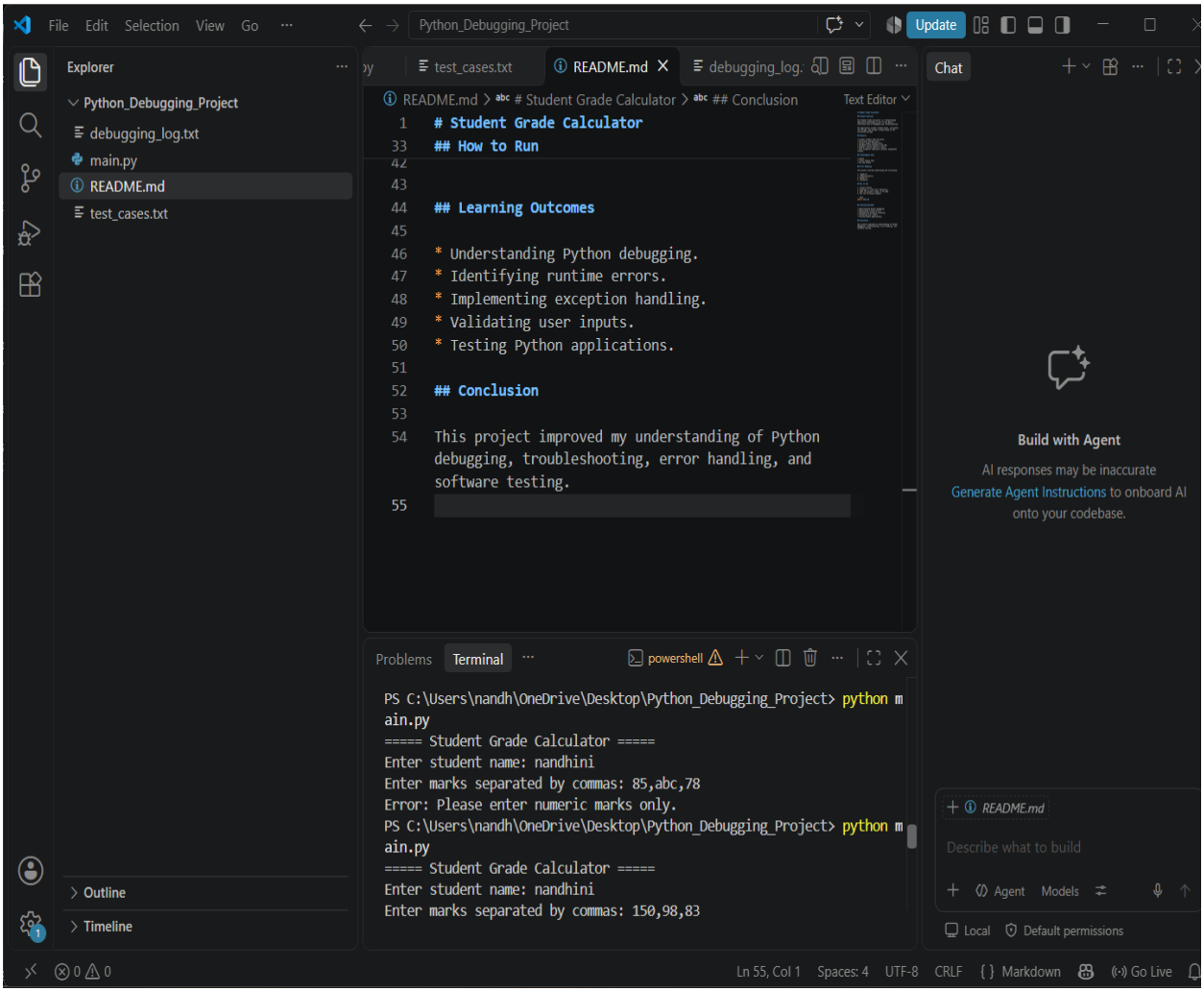


Figure 4:

ValueError – Handling invalid non-numeric input using exception handling.

Bug 5: Invalid Marks

Problem:
The application initially accepted negative marks and marks above 100.

Solution:
Implemented validation using conditional statements and the all() function.

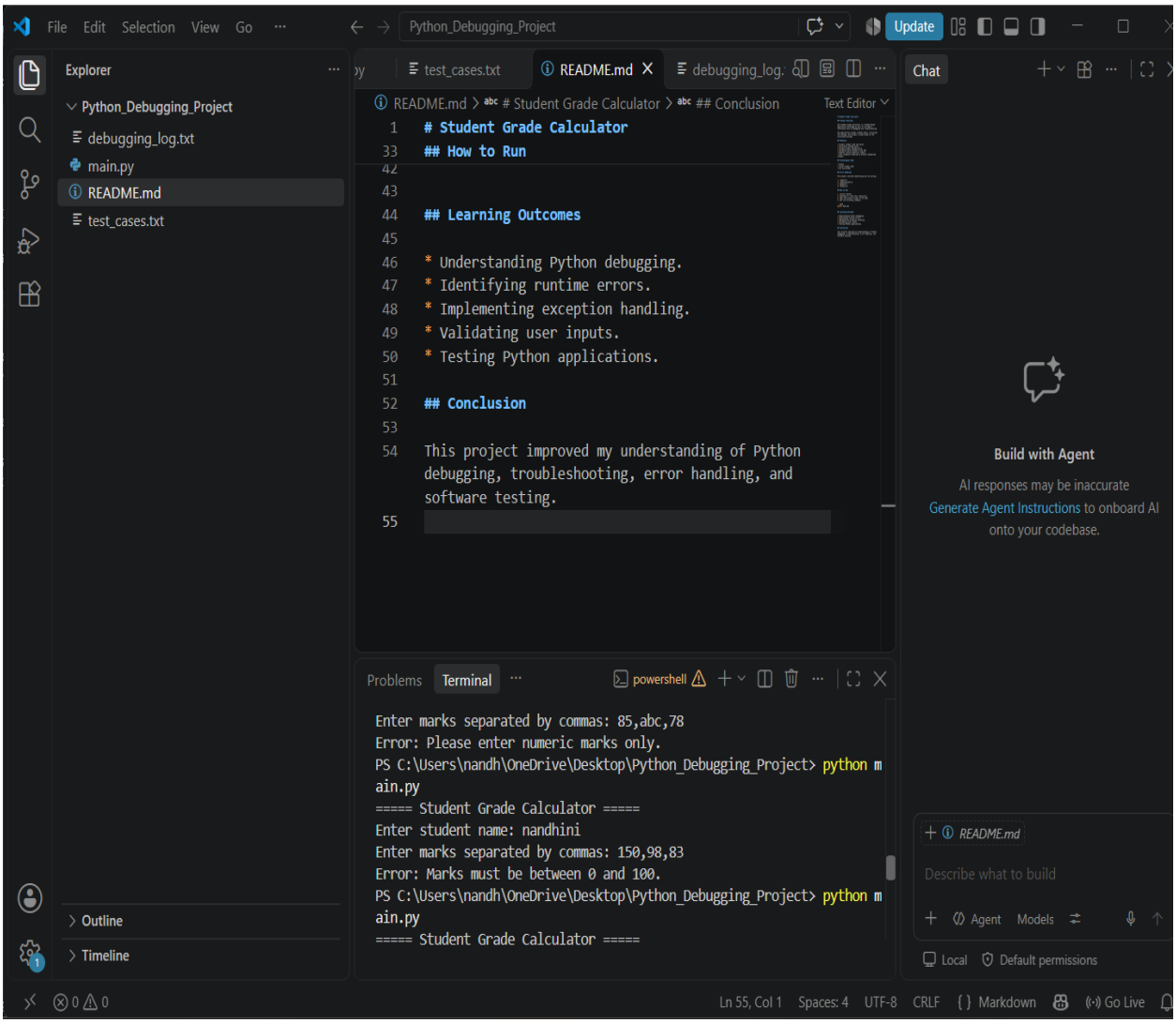


Figure 5:

Input Validation – Detection of marks outside the permitted range.

6. Worked Example

Input:

Student Name: Nandhini

Marks: 85, 90, 78

Calculation:

Total = 85 + 90 + 78

Total = 253

Average = 253 / 3

Average = 84.33

Grade = B

Expected Output:

Student Name: Nandhini

Average Marks: 84.33

Grade: B

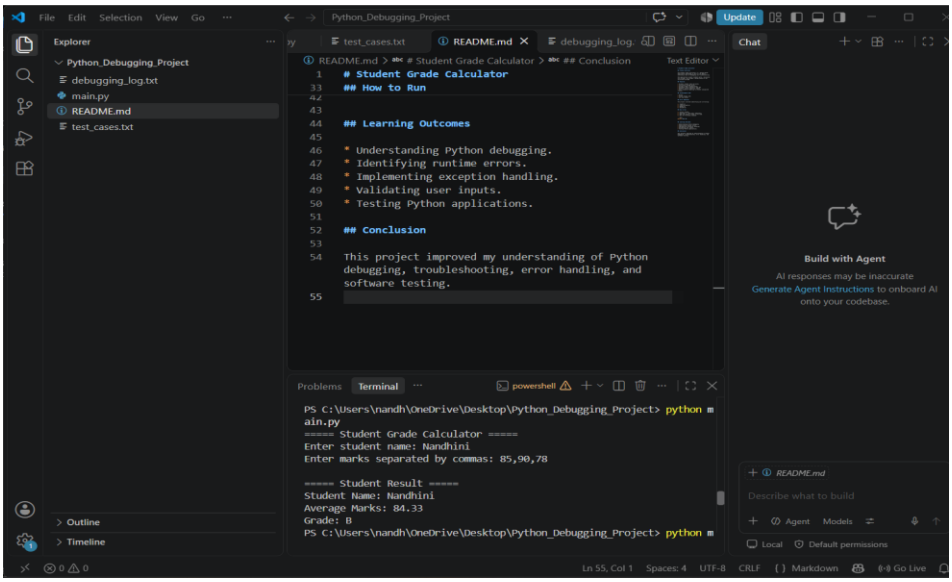


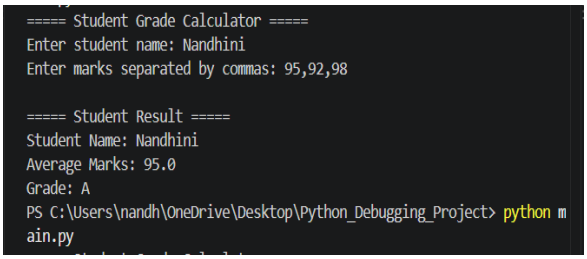
Figure 6:

Successful Execution – Verification of the corrected Student Grade Calculator.

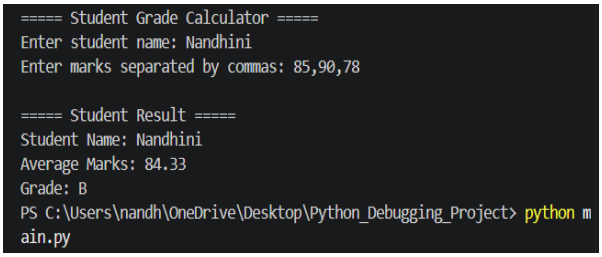
7. Testing

The application is designed to handle the following test scenarios:

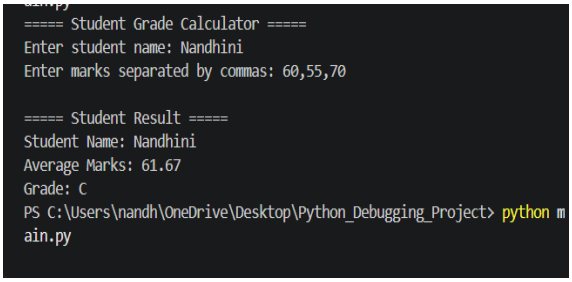
1. Valid marks.
2. Marks belonging to Grade A.



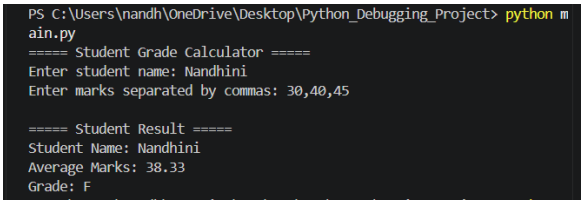
3. Marks belonging to Grade B.



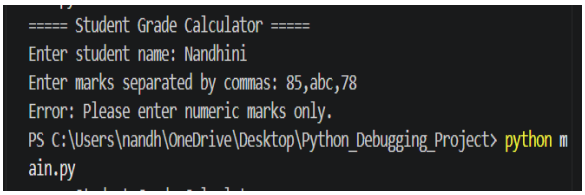
4. Marks belonging to Grade C.



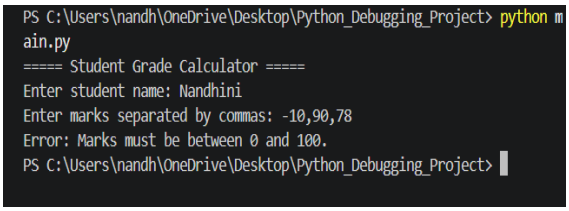
5. Marks belonging to Grade F



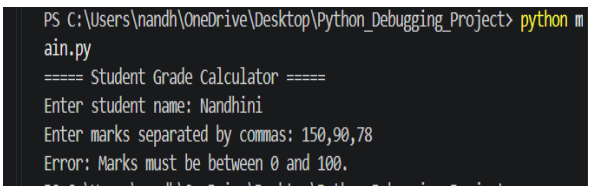
6. Alphabetic input



7. Negative marks



8. Marks exceeding 100.



8. Learning Outcomes

Through this project, I gained practical experience in:

- Identifying and resolving Python errors.
- Understanding TypeError, NameError, IndentationError and ValueError.
- Implementing exception handling.
- Performing input validation.
- Writing reusable functions.
- Testing applications with different inputs.
- Improving code readability and reliability.

9. Conclusion

The Student Grade Calculator project provided practical experience in Python debugging and troubleshooting.

By identifying programming errors, understanding their causes, and implementing appropriate corrections, I improved my problem-solving and programming skills.

This task helped me understand the importance of debugging, input validation, exception handling, and systematic software testing.